

Report della Creazione dell'Applicativo ”Crea_Report”

Alessandro Brocchi

June 2026

Contents

1	Analisi del problema	3
2	Obiettivo del Software	4
3	Struttura dell'Applicativo	5
3.1	Architettura del software	6
3.2	Tecnologie utilizzate	6
3.3	Flusso di elaborazione	7
3.4	Cartella <code>script</code>	7
3.4.1	File: <code>dbf_to_df.py</code>	7
3.4.2	File: <code>df_to_excel.py</code>	10
3.4.3	File: <code>df_format.py</code>	13
3.4.4	File: <code>df_compress.py</code>	15
3.4.5	File: <code>report_config.py</code>	15
3.4.6	File: <code>main.py</code>	16
3.5	Cartella <code>docs</code>	24
3.5.1	File: <code>Manuale_Software_Creazione_Report.pdf</code>	24
4	Cartella release	25
4.1	Cartella <code>config</code>	25
4.1.1	File: <code>report_config.json</code>	25
4.1.2	File: <code>config_cava.txt</code>	27
4.2	Cartella <code>export</code>	28
4.2.1	File di esempio	28
4.3	Cartella <code>_internal</code>	28
5	Utilizzo del Software	29
5.1	Avvio manuale	29
5.2	Esecuzione automatica	29
6	Risoluzione dei problemi	30
6.1	File DBF non trovati	30
6.2	File di configurazione mancanti	30
6.3	Report non generato	30

<i>CONTENTS</i>	2
6.4 CAVA_NON_MAPPATA	30
6.5 Errore durante l'avvio	30
7 Manutenzione ed estensioni future	31

Chapter 1

Analisi del problema

L'azienda dispone di un sistema gestionale basato su database FoxPro dal quale vengono registrate quotidianamente tutte le movimentazioni relative a clienti, articoli, documenti e cave. Sebbene tali informazioni siano già presenti all'interno del sistema informativo aziendale, l'estrazione dei dati necessari per le attività di controllo e rendicontazione richiedeva numerosi passaggi manuali.

La generazione dei report mensili avveniva infatti attraverso interrogazioni specifiche sulle tabelle del database, seguite da operazioni di esportazione, filtraggio e riorganizzazione dei dati. Questo processo comportava tempi elevati di elaborazione, una forte dipendenza dalle conoscenze tecniche dell'operatore e un elevato rischio di errori dovuti a modifiche manuali effettuate sui fogli di calcolo.

Un'ulteriore criticità era rappresentata dalla presenza delle informazioni distribuite su più tabelle correlate tra loro. Per ottenere un report completo era necessario combinare dati provenienti da documenti, righe documento, anagrafiche clienti e tabelle di supporto, ricostruendo ogni volta le relazioni necessarie. Tale attività risultava ripetitiva e difficilmente scalabile nel caso di nuovi report o nuove esigenze informative.

L'obiettivo del progetto è stato quindi quello di realizzare un applicativo in grado di automatizzare l'intero processo di generazione dei report, dalla lettura dei dati presenti nei file DBF fino alla produzione dei file Excel finali. Il sistema doveva essere sufficientemente flessibile da consentire la modifica dei report attraverso semplici file di configurazione, evitando interventi sul codice sorgente per le variazioni più comuni.

Particolare attenzione è stata dedicata alle prestazioni, poiché i database aziendali contengono una grande quantità di record e vengono consultati direttamente dalla rete aziendale. Per questo motivo il software è stato progettato per leggere esclusivamente i campi necessari, applicare filtri già durante la fase di importazione e ridurre il volume dei dati elaborati nelle fasi successive.

Il risultato finale è un sistema modulare che consente di generare automaticamente report periodici standardizzati, riducendo i tempi di elaborazione, minimizzando il rischio di errore umano e semplificando la manutenzione e l'estensione futura delle funzionalità.

Chapter 2

Obiettivo del Software

L'applicativo **Crea.Report** è stato sviluppato con l'obiettivo di automatizzare la generazione dei report periodici utilizzati da ICR per l'analisi delle movimentazioni di magazzino e dei relativi costi associati alle cave.

Prima dello sviluppo del software, la produzione di tali report richiedeva l'estrazione manuale dei dati dal sistema gestionale aziendale, l'esecuzione di filtri e controlli sui dati esportati e la successiva riorganizzazione all'interno di fogli di calcolo. Questo processo comportava tempi elevati di lavorazione e una significativa possibilità di errore umano.

L'obiettivo principale del progetto è stato quindi quello di realizzare un sistema in grado di automatizzare completamente la lettura dei dati, la loro elaborazione e la produzione dei file Excel finali. Parallelamente si è scelto di adottare una struttura configurabile tramite file esterni, così da consentire la modifica dei report senza intervenire direttamente sul codice sorgente.

Il risultato ottenuto è un'applicazione modulare, facilmente estendibile e predisposta alla generazione automatica di nuovi report in funzione delle future esigenze aziendali.

Chapter 3

Struttura dell'Applicativo

A seguito dell'analisi del problema e delle prime prove effettuate sui dati aziendali, il lavoro si è concentrato sulla riorganizzazione e sul consolidamento dei moduli Python sviluppati nelle fasi precedenti. L'obiettivo principale è stato quello di trasformare gli script iniziali in un applicativo più strutturato, in grado di generare automaticamente report mensili relativi ai prodotti e alle movimentazioni di ICR.

Il software realizzato consente di leggere i dati provenienti dalle tabelle Fox-Pro collegate al database aziendale `Private.DBC`, utilizzando una procedura di lettura ottimizzata rispetto alla libreria `dbfread`. In questo modo il programma può accedere alle informazioni necessarie senza caricare interamente le tabelle, migliorando le prestazioni e riducendo il consumo di memoria.

La logica di funzionamento dell'applicativo è guidata da un file di configurazione in formato `JSON`, attraverso il quale vengono definite le colonne da importare, i nomi da assegnare ai campi nel report finale e le regole di elaborazione da applicare durante la costruzione del dataset. Questa scelta rende il sistema più flessibile, poiché eventuali modifiche alla struttura del report possono essere gestite intervenendo sulla configurazione, senza modificare direttamente il codice sorgente.

Una volta lette ed elaborate le tabelle `DBF`, il programma costruisce un *DataFrame* contenente i dati necessari alla generazione dei report. La procedura è attualmente impostata per produrre report mensili relativi al periodo corrente e ai due mesi precedenti, generando tre file Excel distinti. Ogni file viene costruito secondo le regole definite nella configurazione e contiene le principali informazioni utili all'analisi, tra cui il tipo documento, il periodo di riferimento, il gruppo magazzino, il codice articolo, la descrizione del prodotto, l'unità di misura, la quantità, il valore totale, la cava di provenienza, il codice cliente o fornitore e la ragione sociale.

Il risultato ottenuto è quindi una prima versione funzionante dell'applicativo, capace di automatizzare una parte significativa del processo di estrazione, normalizzazione e produzione dei report mensili, mantenendo al tempo stesso una struttura configurabile e predisposta a future estensioni.

3.1 Architettura del software

Per rendere più chiara la struttura dell'applicativo e facilitarne la manutenzione futura, il progetto è stato organizzato secondo una suddivisione modulare delle funzionalità. Ogni cartella raccoglie componenti con responsabilità specifiche, separando la logica di elaborazione dei dati dalla configurazione, dalla documentazione e dai risultati prodotti dal software.

Nei paragrafi successivi verranno analizzate le principali cartelle che compongono il Progetto **Crea_Report**, descrivendone il ruolo all'interno dell'architettura complessiva e illustrando, ove opportuno, il funzionamento dei moduli più significativi. Questa organizzazione consente di semplificare lo sviluppo di nuove funzionalità, favorisce il riutilizzo del codice e rende più agevole l'estensione dell'applicativo a nuove tipologie di report.

3.2 Tecnologie utilizzate

Lo sviluppo del progetto è stato realizzato utilizzando tecnologie open source largamente diffuse nel settore dell'elaborazione dati.

- Python come linguaggio principale dell'applicativo;
- Pandas per la gestione e l'elaborazione dei DataFrame;
- OpenPyXL per la generazione e la formattazione dei file Excel;
- Database FoxPro come sorgente dei dati aziendali;
- JSON per la configurazione dinamica dei report;
- PyInstaller per la generazione dell'eseguibile distribuito;
- Windows Task Scheduler per l'automazione delle esecuzioni periodiche.

La scelta di tali tecnologie è stata effettuata privilegiando affidabilità, semplicità di manutenzione e possibilità di estensione futura del software.

3.3 Flusso di elaborazione

L'elaborazione dei dati segue una pipeline composta da più fasi consecutive. I dati vengono inizialmente letti dai file DBF aziendali e trasformati in DataFrame Pandas. Successivamente vengono applicati i filtri configurati, calcolati i campi derivati e realizzate le eventuali aggregazioni richieste dal report. L'ultima fase consiste nella generazione e formattazione dei file Excel destinati agli utenti finali.



3.4 Cartella script

La cartella `script` costituisce il nucleo operativo dell'applicativo, ma è anche l'unica che non è presente nella cartella dell'applicativo. Questo perché l'applicativo può essere utilizzato anche con config differenti. Di conseguenza questa cartella è a disposizione dei tecnici che sanno come comprenderla. Al suo interno sono raccolti i moduli Python che gestiscono le principali fasi del processo: lettura dei file DBF, costruzione e trasformazione dei *DataFrame*, applicazione delle regole di elaborazione, aggregazione dei dati e generazione dei report Excel finali. La suddivisione in file distinti permette di separare le diverse responsabilità del programma, rendendo il codice più leggibile, riutilizzabile e facilmente manutenibile.

3.4.1 File: `dbf_to_df.py`

Importiamo le librerie:

```
1 from datetime import date
2 from pathlib import Path
3 import struct
4
5 import pandas as pd
```


Definisco una funzione ”leggi_campi_DBF()”:

```

6
7 def _leggi_campi_dbf(f):
8     f.seek(4)
9     num_records = struct.unpack("<I", f.read(4))[0]
10    header_len = struct.unpack("<H", f.read(2))[0]
11    record_len = struct.unpack("<H", f.read(2))[0]
12    f.seek(32)
13
14    campi = {}
15    while True:
16        raw = f.read(32)
17        if not raw or raw[0] == 0x0D:
18            break
19
20        nome = raw[:11].split(b"\x00", 1)[0].decode("ascii").strip()
21        campi[nome] = {
22            "tipo": chr(raw[11]),
23            "lunghezza": raw[16],
24            "decimali": raw[17],
25        }
26
27        offset = 1
28        for campo in campi.values():
29            campo["offset"] = offset
30            offset += campo["lunghezza"]
31
32    return num_records, header_len, record_len, campi

```

Questa funzione ha il compito di leggere la struttura interna di un file DBF prima dell'importazione dei dati veri e propri. In particolare recupera le informazioni principali del file, come il numero di record presenti, la dimensione dell'intestazione e la lunghezza di ciascun record. Successivamente analizza la descrizione dei campi contenuti nella tabella, individuando per ognuno il nome, il tipo di dato, la lunghezza e l'eventuale numero di decimali. Infine calcola la posizione di ogni campo all'interno del record, così da permettere alle funzioni successive di leggere correttamente i valori grezzi presenti nel file.

Definiamo una Funzione ”_parse_valore()”:

```

33 def _parse_valore(raw, tipo, decimali):
34     if tipo == "C":
35         return raw.decode("cp1252", errors="replace").strip()
36
37     if tipo == "I":
38         if raw in {b"\x00\x00\x00\x00", b"uuuu"}:
39             return None
40         return struct.unpack("<i", raw[:4])[0]
41
42     if tipo == "B":
43         if raw == b"\x00" * len(raw):
44             return 0.0
45         return struct.unpack("<d", raw[:8])[0]
46
47     if tipo == "Y":
48         if raw == b"\x00" * len(raw):
49             return 0.0
50         return struct.unpack("<q", raw[:8])[0] / 10000
51
52     raw = raw.strip()
53     if not raw:
54         return None
55
56     if tipo == "D":
57         testo = raw.decode("ascii", errors="ignore")
58         if len(testo) != 8:
59             return None
60         return date(int(testo[:4]), int(testo[4:6]), int(testo[6:8]))
61
62     if tipo in {"N", "F"}:
63         testo = raw.decode("ascii", errors="ignore").strip()
64         if not testo:
65             return None
66         try:
67             return float(testo) if decimali else int(float(testo))
68         except ValueError:
69             return None
70
71     if tipo == "L":
72         return raw[:1].upper() in {b"T", b"Y"}
73
74     return raw.decode("cp1252", errors="replace").strip()

```

Questa funzione ha il compito di convertire i valori grezzi letti dal file DBF in dati correttamente interpretabili dal linguaggio Python. In base al tipo di campo individuato nella struttura della tabella, effettua le opportune trasformazioni per ottenere stringhe di testo, numeri interi, numeri decimali, date o valori logici. La funzione gestisce inoltre eventuali campi vuoti, valori nulli o formati non validi, garantendo che i dati restituiti siano coerenti e utilizzabili nelle successive fasi di elaborazione. In questo modo rappresenta il collegamento tra la rappresentazione binaria presente nel file DBF e le strutture dati utilizzate dal programma.

Definiamo una Funzione ”_leggi_record()”:

```

76
77 def _leggi_record(record, nomi_campi, campi):
78     valori = {}
79     for nome in nomi_campi:
80         campo = campi.get(nome)
81         if campo is None:
82             continue
83
84         start = campo["offset"]
85         end = start + campo["lunghezza"]
86         valori[nome] = _parse_valore(
87             record[start:end],
88             campo["tipo"],
89             campo["decimali"],
90         )
91     return valori
92

```

Questa funzione si occupa di estrarre i dati contenuti in un singolo record del file DBF. Utilizzando le informazioni sulla struttura della tabella ottenute in precedenza, individua la posizione e la dimensione di ciascun campo richiesto, legge il valore corrispondente dal record e lo converte in un formato comprensibile per Python. Al termine dell'elaborazione restituisce una raccolta di valori associati ai rispettivi nomi di campo, pronta per essere utilizzata nelle fasi successive di costruzione del DataFrame.

Definiamo la Funzione ”esegui()”:

```

94
95 def esegui(percorso_dbf, col_u, msg, filtro=None):
96     records = []
97
98     with Path(percorso_dbf).open("rb") as f:
99         num_records, header_len, record_len, campi = _leggi_campi_dbf(f)
100         nomi_campi = [col for col in col_u if col in campi]
101         f.seek(header_len)
102
103         for _ in range(num_records):
104             record_raw = f.read(record_len)
105             if not record_raw:
106                 break
107             if record_raw[:1] == b"*":
108                 continue
109
110             record = _leggi_record(record_raw, nomi_campi, campi)
111             if filtro is not None and not filtro(record):
112                 continue
113
114             records.append({
115                 f"{col}_{msg}": record[col]
116                 for col in nomi_campi
117             })
118
119     return pd.DataFrame.from_records(records)

```

È la funzione più importante in quanto racchiude gli utilizzi delle funzioni definite in precedenza. Questa funzione è il core di questo file e la sua funzione definisce anche il nome del file. Essa infatti analizza, attraverso `leggi_campi_dbf()` la struttura dei dati all'interno del dbf; dopodiché inizia a filtrare le colonne della tabella necessarie, e gli eventuali filtri sui valori che possono essere stati passati dal main, attraverso `_leggi_record()` che richiama al suo interno `_parse_valori()`. Al termine restituisce al programma `main.py` che l'ha richiamata, il DataFrame Pandas richiesto.

3.4.2 File: df_to_excel.py

Importiamo le librerie necessarie:

```

1 from datetime import datetime
2 from pathlib import Path
3 import sys
4
5 from openpyxl import load_workbook
6 from openpyxl.formatting.rule import CellIsRule
7 from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
8 from openpyxl.utils import get_column_letter
9
10 if getattr(sys, "frozen", False):
11     BASE_DIR = Path(sys.executable).resolve().parent
12 else:
13     BASE_DIR = Path(__file__).resolve().parent.parent
14
15 EXPORT_DIR = BASE_DIR / "export"
16 EXPORT_PATH = EXPORT_DIR / "report.xlsx"

```

Questa parte iniziale prepara il modulo dedicato alla generazione e alla formattazione dei file Excel. Vengono importati gli strumenti necessari per aprire e modificare workbook, applicare stili grafici, gestire formati condizionali, bordi, colori, allineamenti e larghezze delle colonne. Successivamente viene individuata la cartella principale del progetto, distinguendo tra esecuzione come script Python ed esecuzione come applicativo compilato. Infine vengono definiti i percorsi di esportazione, cioè la cartella in cui verranno salvati i report e il nome predefinito del file Excel prodotto.

Definiamo alcune Funzioni grafiche:

```

17 def _normalizza_nome_colonna(nome):
18     return str(nome).replace("-", "_").title()
19
20
21 def _stile_intestazione(ws):
22     header_fill = PatternFill("solid", fgColor="173F35")
23     header_font = Font(color="FFFFFF", bold=True)
24     thin_border = Border(bottom=Side(style="thin", color="D7DEDA"))
25
26     for cell in ws[1]:
27         cell.fill = header_fill
28         cell.font = header_font
29         cell.alignment = Alignment(horizontal="center", vertical="center")
30         cell.border = thin_border
31
32     ws.row_dimensions[1].height = 24

```

Queste funzioni hanno il compito di migliorare la leggibilità del report Excel generato. La prima si occupa di trasformare i nomi tecnici delle colonne in etichette più chiare e facilmente comprensibili per l'utente finale, rendendo il report più intuitivo da consultare. La seconda definisce invece l'aspetto grafico delle intestazioni, applicando uno stile uniforme caratterizzato da colori, grassetto, allineamento e bordi. Viene inoltre impostata un'altezza specifica per la riga delle intestazioni, così da garantire una presentazione più ordinata e professionale dei dati esportati.

Definiamo una Funzione ”_formatta_foglio_principale()”:

```

33
34 def _formatta_foglio_principale(ws, config_report):
35     ws.title = config_report.get("main_sheet", "Report")
36     ws.sheet_view.showGridLines = False
37     ws.freeze_panes = config_report.get("freeze_panes", "A2")
38
39     original_headers = {cell.value: cell.column for cell in ws[1]}
40     larghezze = config_report.get("column_widths", {})
41     wrap_columns = set(config_report.get("wrap_columns", []))
42     number_formats = config_report.get("number_formats", {})
43     soft_fill = PatternFill("solid", fgColor="F6F8F7")
44     thin_border = Border(bottom=Side(style="thin", color="D7DEDA"))
45
46     for cell in ws[1]:
47         nome_originale = cell.value
48         ws.column_dimensions[cell.column_letter].width = larghezze.get(nome_originale, 18)
49         cell.value = _normalizza_nome_colonna(nome_originale)
50
51     _stile_intestazione(ws)
52
53     for row in ws.iter_rows(min_row=2):
54         if row[0].row % 2 == 0:
55             for cell in row:
56                 cell.fill = soft_fill
57
58         for cell in row:
59             cell.border = thin_border
60             cell.alignment = Alignment(vertical="center")
61
62         for colonna in wrap_columns:
63             col_index = original_headers.get(colonna)
64             if col_index:
65                 row[col_index - 1].alignment = Alignment(wrap_text=True, vertical="center")
66
67     for colonna, formato in number_formats.items():

```

```

68         col_index = original_headers.get(colonna)
69         if col_index:
70             row[col_index - 1].number_format = formato
71
72     val_tot_col = original_headers.get("Val_tot")
73     if val_tot_col and ws.max_row > 1:
74         lettera = get_column_letter(val_tot_col)
75         ws.conditional_formatting.add(
76             f"{lettera}2:{lettera}{ws.max_row}",
77             CellIsRule(operator="equal", formula=["0"], font=Font(color="7A817D")),
78         )

```

Questa funzione si occupa di rendere il foglio principale del report Excel più leggibile, ordinato e pronto per l'utilizzo finale. Inizialmente assegna al foglio il nome previsto dalla configurazione, nasconde la griglia visiva di Excel e blocca la prima riga, così che le intestazioni restino sempre visibili durante lo scorrimento. Successivamente legge le impostazioni relative alla larghezza delle colonne, ai formati numerici e alle colonne che devono andare a capo automaticamente, mantenendo memoria dei nomi originali delle intestazioni per applicare correttamente le regole di formattazione. Le intestazioni vengono poi rinominate in una forma più chiara e formattate graficamente, mentre le righe dei dati vengono rese più leggibili tramite un'alternanza cromatica, bordi leggeri e allineamento verticale centrato. Per alcune colonne specifiche viene attivato il testo a capo, utile quando i contenuti sono lunghi, mentre per le colonne numeriche vengono applicati i formati previsti, ad esempio per importi, quantità o valori percentuali. Infine, sulla colonna del valore totale viene applicata una regola di formattazione condizionale che rende meno evidenti i valori pari a zero, migliorando la leggibilità complessiva del report e aiutando l'utente a concentrarsi sui dati realmente significativi.

Definiamo una Funzione ”_scrivi_riepilogo()”:

```

79
80 def _scrivi_riepilogo(wb, df, summary_config, index):
81     sheet_name = summary_config.get("sheet_name", f"Riepilogo_{index}")
82     if sheet_name in wb.sheetnames:
83         del wb[sheet_name]
84
85     group_by = summary_config.get("group_by", [])
86     aggregazioni = summary_config.get("aggregations", {})
87     if not group_by or not aggregazioni:
88         return
89
90     colonne = group_by + list(aggregazioni)
91     riepilogo = {
92         df.groupby(group_by, dropna=False, as_index=False)
93         .agg(aggregazioni)
94         .sort_values(group_by, kind="stable")
95     }
96
97     ws = wb.create_sheet(sheet_name)
98     ws.sheet_view.showGridlines = False
99     ws.freeze_panes = "A2"
100    ws.append([_normalizza_nome_colonna(colonna) for colonna in colonne])
101
102    for _, row in riepilogo[colonne].iterrows():
103        ws.append(row.tolist())
104
105    _stile_intestazione(ws)
106    thin_border = Border(bottom=Side(style="thin", color="D7DEDA"))
107    number_formats = summary_config.get("number_formats", {})
108
109    for cell in ws[1]:
110        ws.column_dimensions[cell.column_letter].width = 18
111
112    for row in ws.iter_rows(min_row=2):
113        for cell in row:
114            cell.border = thin_border
115            cell.alignment = Alignment(vertical="center")
116
117        for posizione, colonna in enumerate(colonne):
118            formato = number_formats.get(colonna)
119            if formato:
120                row[posizione].number_format = formato

```

Questa funzione ha il compito di generare un foglio di riepilogo all'interno del report Excel finale partendo dai dati già elaborati. Dopo aver verificato che non esista già un foglio con lo stesso nome, utilizza le regole definite nella configurazione per raggruppare e aggregare le informazioni secondo i criteri richiesti. Il risultato ottenuto viene ordinato e riportato in un nuovo foglio di lavoro, nel quale vengono inserite intestazioni più leggibili e tutti i dati sintetici derivanti dall'elaborazione. Successivamente vengono applicate una serie di formattazioni grafiche e funzionali, come il blocco della prima riga, l'adattamento della larghezza delle colonne, l'allineamento delle celle, l'aggiunta di bordi e l'applicazione di eventuali formati numerici specifici. In questo modo il foglio prodotto fornisce una vista aggregata e facilmente consultabile dei dati presenti nel report principale, consentendo all'utente di analizzare rapidamente totali, raggruppamenti e indicatori di interesse senza dover effettuare ulteriori elaborazioni manuali in Excel.

Definiamo la Funzione ”_formatta_excel()”:

```

122 def _formatta_excel(percorso, df, config_report):
123     wb = load_workbook(percorso)
124     _formatta_foglio_principale(wb.active, config_report)
125
126     for index, summary_config in enumerate(config_report.get("summaries", []), start=1):
127         _scrivi_riepilogo(wb, df, summary_config, index)
128
129     wb.active = wb.sheetnames.index(config_report.get("main_sheet", "Report"))
130     wb.save(percorso)

```

Questa funzione è l'ultimo passaggio della generazione del report Excel. Fa sostanzialmente cinque cose:

1. Apre il file Excel che è stato appena creato.
2. Formatta il foglio principale, cioè quello che contiene tutti i dati dettagliati del report (larghezza colonne, intestazioni, formati numerici, filtri, ecc.).
3. Crea i fogli di riepilogo definiti nella configurazione.
Se nel file di configurazione hai uno o più "summaries", li percorre uno alla volta e genera i rispettivi fogli riassuntivi.
4. Imposta il foglio principale come foglio iniziale che l'utente vedrà quando aprirà il file.
5. Salva il file Excel con tutte le modifiche.

Definiamo un Funzione ”_percorso_export()”:

```

131 def _percorso_export(config_report):
132     nome_file = config_report.get("output_file", EXPORT_PATH.name) if config_report else EXPORT_PATH.name
133     nome_file = nome_file.format(**config_report)
134     return EXPORT_DIR / nome_file

```

Questa funzione Serve a decidere nome e posizione del file Excel finale. Prende il nome del File dalla configurazione, se non lo trova mette il predefinito, sostituisce eventuali parti dinamiche, e costruisce il percorso completo dentro la cartella di esportazione.

Definiamo un Funzione ”esegui()”:

```

135 def esegui(df, config_report=None):
136     config_report = config_report or {}
137     EXPORT_DIR.mkdir(parents=True, exist_ok=True)
138     percorso_base = _percorso_export(config_report)
139
140     try:
141         percorso = percorso_base
142         df.to_excel(percorso, index=False)
143         _formatta_excel(percorso, df, config_report)
144     except PermissionError:
145         percorso = percorso_base.with_name(
146             f"{percorso_base.stem}_{datetime.now().strftime('%Y%m%d_%H%M%S')}{percorso_base.suffix}"
147         )
148         df.to_excel(percorso, index=False)
149         _formatta_excel(percorso, df, config_report)
150         print(f"Report principale bloccato. Creata una copia: {percorso}")
151         return percorso
152
153     print(f"Conversione completata. File creato: {percorso}")
154     return percorso
155

```

Questa funzione rappresenta il punto finale del processo di generazione del report. Il suo compito è preparare la cartella di destinazione, determinare il nome e il percorso da generare, esportare come formato Excel. Deve saper anche gestire situazioni in cui il file risulta aperto o bloccato da un altro programma. In questo caso, invece di interrompere l'esecuzione, genera una copia del report con un nome univoco basato su data e ora, garantendo il completamento dell'esportazione.

3.4.3 File: df_format.py

Importiamo le librerie:

```
1 import pandas as pd
2 import dbf_to_df
```

Definiamo la funzione ”_colonna_con_suffix():

```
3 def _colonna_con_suffix(colonna, suffix):
4     return f"{colonna}_{suffix}"
```

Questa funzione ha il compito di generare automaticamente un nome di colonna univoco associando al nome originale un suffisso identificativo. Tale meccanismo viene utilizzato per distinguere campi provenienti da tabelle diverse che potrebbero avere lo stesso nome, evitando ambiguità durante le operazioni di unione e di elaborazione dei dati. In questo modo il programma è in grado di mantenere traccia dell'origine delle informazioni e di gestire correttamente dataset costruiti a partire da più sorgenti collegate tra loro.

Definiamo la funzione ”_leggi_sorgente():

```
5 def _leggi_sorgente(nome, source_config, base_dir, menno, dataframes):
6     columns = source_config.get("columns", [])
7     suffix = source_config.get("suffix", "")
8     percorso = base_dir / source_config["file"]
9
10    filtro_data = source_config.get("date_filter")
11    filtro_in = source_config.get("filter_in")
12
13    def filtro(record):
14        if filtro_data:
15            valore = record.get(filtro_data["column"])
16            if not (pd.Timestamp(menno[1]) <= pd.Timestamp(valore) <= pd.Timestamp(menno[2])):
17                return False
18        if filtro_in:
19            source_name = filtro_in["source"]
20            source_column = _colonna_con_suffix(
21                filtro_in["source_column"],
22                dataframes[source_name]["suffix"],
23            )
24            valori_validi = dataframes[source_name]["values"][source_column]
25            if record.get(filtro_in["column"]) not in valori_validi:
26                return False
27        return True
28
29    df = dbf_to_df.esegui(
30        percorso,
31        columns,
32        suffix,
33        filtro=filtro if filtro_data or filtro_in else None,
34    )
35
36    value_sets = {
37        colonna: set(df[colonna])
38        for colonna in df.columns
39    }
40
41    return {
42        "df": df,
43        "suffix": suffix,
44        "values": value_sets,
45    }
46
```

Questa funzione si occupa di leggere una sorgente dati e trasformarla in un DataFrame pronto per le elaborazioni successive. Durante la lettura applica eventuali filtri definiti nella configurazione, come la selezione di un determinato intervallo temporale o il mantenimento delle sole righe collegate ai risultati ottenuti da sorgenti già elaborate in precedenza. Una volta completata l'importazione, costruisce anche una raccolta dei valori unici presenti nelle diverse colonne, che verrà utilizzata nelle fasi successive per effettuare ulteriori filtri o collegamenti tra tabelle. Infine restituisce sia il DataFrame contenente i dati letti sia le informazioni ausiliarie necessarie per le elaborazioni successive della pipeline.

Definiamo la funzione "esegui()":

```
48 def esegui(base_dir, dataset_config, menno):
49     dataframes = {}
50     sources = dataset_config.get("sources", {})
51
52     for nome, source_config in sources.items():
53         dataframes[nome] = _leggi_sorgente(nome, source_config, base_dir, menno, dataframes)
54
55     base_source = dataset_config["base_source"]
56     df = dataframes[base_source]["df"]
57
58     for join_config in dataset_config.get("joins", []):
59         source_name = join_config["source"]
60         df = pd.merge(
61             df,
62             dataframes[source_name]["df"],
63             left_on=join_config["left_on"],
64             right_on=join_config["right_on"],
65             how=join_config.get("how", "left"),
66         )
67
68     return df
```

Questa funzione coordina l'intero processo di costruzione del dataset finale. Inizialmente legge tutte le sorgenti dati previste dalla configurazione, applicando i filtri necessari e memorizzando le informazioni ottenute. Successivamente individua la sorgente principale che costituirà la base del dataset e, partendo da essa, integra progressivamente i dati provenienti dalle altre sorgenti attraverso una serie di collegamenti definiti nella configurazione. Al termine di questo processo restituisce un unico DataFrame contenente tutte le informazioni necessarie, già unite e pronte per le elaborazioni e la generazione del report finale.

3.4.4 File: df_compress.py

Definisco la funzione "esegui()":

```

1 def esegui(df, group_by, aggregazioni=None):
2     if not group_by:
3         return df.copy()
4
5     funzioni_aggregazione = aggregazioni or {
6         "Descrizione_Riga": "first",
7         "Quantita": "sum",
8         "Val_tot": "sum",
9         "Cava": "first",
10        "Ragione_Sociale": "first",
11    }
12
13    df_compresso = (
14        df.groupby(group_by, as_index=False)
15        .agg(funzioni_aggregazione)
16    )
17
18    return df_compresso

```

Questa funzione ha il compito di ridurre e sintetizzare il dataset attraverso un processo di raggruppamento. Se non viene specificato alcun criterio di aggregazione, restituisce semplicemente una copia dei dati originali; in caso contrario, raggruppa le righe secondo le colonne indicate e applica una serie di operazioni sulle informazioni contenute nel dataset. I valori numerici vengono aggregati tramite somme, mentre per le informazioni descrittive viene mantenuto un valore rappresentativo del gruppo. Il risultato è un nuovo DataFrame più compatto, nel quale più record vengono condensati in un'unica riga riassuntiva, mantenendo le informazioni necessarie per l'analisi e riducendo il volume complessivo dei dati da esportare o elaborare successivamente.

3.4.5 File: report_config.py

Importo le librerie e definisco la funziona "carica()":

```

1 import json
2 def carica(percorso, nome_report=None):
3     with open(percorso, "r", encoding="utf-8") as f:
4         config = json.load(f)
5         report_nome = nome_report or config.get("default_report")
6         reports = config.get("reports", {})
7         if report_nome not in reports:
8             disponibili = ", ".join(sorted(reports))
9             raise ValueError(f"Report '{report_nome}' non configurato. Disponibili: {disponibili}")
10        report = reports[report_nome].copy()
11        dataset_nome = report.get("dataset")
12        datasets = config.get("datasets", {})
13        if dataset_nome not in datasets:
14            disponibili = ", ".join(sorted(datasets))
15            raise ValueError(f"Dataset '{dataset_nome}' non configurato. Disponibili: {disponibili}")
16        report["dataset_config"] = datasets[dataset_nome]
17        report["name"] = report_nome
18        return report

```

Questa funzione serve a caricare la configurazione generale del progetto e a selezionare il report da generare. Legge il file di configurazione, individua il report richiesto oppure quello predefinito, verifica che sia effettivamente presente tra quelli disponibili e, in caso contrario, segnala l'errore indicando le alternative configurate. Dopo aver trovato il report corretto, recupera anche il dataset associato e controlla che sia definito correttamente. Infine unisce le informazioni del report con la configurazione del dataset corrispondente e restituisce una configurazione completa, pronta per essere usata nelle fasi successive di lettura dei dati, elaborazione e generazione del file finale.

3.4.6 File: main.py

Importiamo le Librerie e Definiamo le Directory:

```

1 import argparse
2 import calendar
3 import os
4 import sys
5 from datetime import date, datetime
6 from pathlib import Path
7
8 import pandas as pd
9
10 import df_compress as dc
11 import df_format
12 import df_to_excel
13 import report_config
14
15 if getattr(sys, "frozen", False):
16     BASE_DIR = Path(sys.executable).resolve().parent
17 else:
18     BASE_DIR = Path(__file__).resolve().parent.parent
19
20 CONFIG_DIR = BASE_DIR / "config"
21 ESEMPI_DIR = BASE_DIR / "esempi"
22 RETE_DIR = Path(r"\\172.16.2.13\arca\ditte\ICR")

```

Questa parte iniziale del programma prepara l'ambiente di lavoro dell'applicativo. Vengono importati i moduli necessari per gestire gli argomenti da terminale, le date, i percorsi dei file, i dati tabellari e le diverse funzioni sviluppate nei moduli del progetto. Successivamente viene determinata la cartella principale dell'applicativo, distinguendo il caso in cui il programma venga eseguito come normale script Python da quello in cui venga avviato come eseguibile. Infine vengono definiti i percorsi fondamentali usati dal software, cioè la cartella di configurazione, la cartella degli esempi locali e la cartella di rete aziendale da cui vengono letti i file DBF reali.

Definiamo la Funzione "base_dbf_dir()"

```

23 def base_dbf_dir(sorgente):
24     return RETE_DIR if sorgente == "rete" else ESEMPI_DIR

```

Questa funzione serve a scegliere da dove leggere i file DBF in base alla modalità di esecuzione del programma. Se viene selezionata la sorgente di rete, il software utilizza la cartella aziendale reale; in caso contrario, lavora sulla cartella locale degli esempi. In questo modo lo stesso programma può essere usato sia in produzione, sui dati effettivi dell'azienda, sia in fase di test, senza modificare il resto del codice.

Definiamo la Funzione "verifica_percorsi()"

```

25 def verifica_percorsi(sorgente, dataset_config):
26     base_dir = base_dbf_dir(sorgente)
27     dbf_files = {
28         source_config["file"]
29         for source_config in dataset_config.get("sources", {}).values()
30     }
31     mancanti = [
32         str(base_dir / file_name)
33         for file_name in sorted(dbf_files)
34         if not (base_dir / file_name).exists()
35     ]
36     config_da_verificare = [CONFIG_DIR / "report_config.json"]
37     if "Cava" in set(dataset_config.get("calculated_fields", [])):
38         config_da_verificare.append(CONFIG_DIR / "config_cava.txt")
39
40     config_mancanti = [
41         str(percorso)
42         for percorso in config_da_verificare
43         if not percorso.exists()
44     ]
45
46     if mancanti:
47         raise FileNotFoundError("DBF_mancanti:\n" + "\n".join(mancanti))
48     if config_mancanti:
49         raise FileNotFoundError("Config_mancanti:\n" + "\n".join(config_mancanti))
50
51     print(f"Sorgente_DBF: {base_dir}")
52     print(f"Config_progetto: {CONFIG_DIR}")

```

Questa funzione serve a scegliere da dove leggere i file DBF in base alla modalità di esecuzione del programma. Se viene selezionata la sorgente di rete, il software utilizza la cartella aziendale reale; in caso contrario, lavora sulla cartella locale degli esempi. In questo modo lo stesso programma può essere usato sia in produzione, sui dati effettivi dell'azienda, sia in fase di test, senza modificare il resto del codice.

Definiamo la Funzione "valida_data()"

```

53 def valida_data(stringa_input):
54     try:
55         return datetime.strptime(stringa_input, "%Y-%m").date()
56     except ValueError as exc:
57         raise argparse.ArgumentTypeError("Formato non valido. Usa YYYY-MM.") from exc

```

Questa funzione si occupa di validare le date inserite dall'utente durante l'esecuzione del programma. Il suo obiettivo è verificare che il valore fornito rispetti il formato previsto per l'identificazione di un periodo mensile e convertirlo in una rappresentazione temporale utilizzabile dalle successive elaborazioni. Nel caso in cui il formato non sia corretto, la funzione interrompe l'acquisizione del dato e restituisce un messaggio di errore esplicativo, guidando l'utente verso l'inserimento di un valore valido. In questo modo viene garantita la correttezza delle informazioni temporali utilizzate per filtrare i dati e generare i report.

Definiamo la Funzione "mese_con_offset():"

```

58 def mese_con_offset(data_riferimento, offset_mesi):
59     mese_zero_based = data_riferimento.year * 12 + data_riferimento.month - 1 + offset_mesi
60     anno = mese_zero_based // 12
61     mese = mese_zero_based % 12 + 1
62     return date(anno, mese, 1)

```

Questa funzione ha il compito di calcolare una data corrispondente a un determinato numero di mesi prima o dopo una data di riferimento. Viene utilizzata per gestire in modo corretto il passaggio tra anni e mesi, consentendo di individuare automaticamente il primo giorno del mese risultante. Tale meccanismo è particolarmente utile nella generazione dei report periodici, poiché permette di determinare con precisione i mesi da analizzare senza richiedere calcoli manuali da parte dell'utente.

Definiamo la Funzione "periodo_mese():"

```

63 def periodo_mese(data_riferimento):
64     anno = data_riferimento.year
65     mese = data_riferimento.month
66     menno = data_riferimento.strftime("%m/%Y")
67     ultimo_giorno = calendar.monthrange(anno, mese)[1]
68     data_inizio = data_riferimento.replace(day=1)
69     data_fine = data_riferimento.replace(day=ultimo_giorno)
70     print(f"Mese elaborato: {menno}")
71     print(f"Data inizio: {data_inizio}")
72     print(f"Data fine: {data_fine}")
73     return menno, data_inizio, data_fine
74
75

```

Questa funzione ha il compito di determinare l'intervallo temporale associato a un determinato mese. A partire da una data di riferimento, ricava l'anno e il mese corrispondenti, individua automaticamente il primo e l'ultimo giorno del periodo e genera una rappresentazione sintetica del mese utilizzata nelle successive elaborazioni. Inoltre fornisce all'utente un riscontro delle date effettivamente considerate dal programma, facilitando il controllo dei dati che verranno inclusi nella generazione del report. Il risultato finale è costituito dalle informazioni necessarie per delimitare correttamente il periodo di analisi mensile.

Definiamo la Funzione "periodo_da_elaborare():"

```

76 def periodi_da_elaborare(numero_mesi=3, oggi=None):
77     oggi = oggi or date.today()
78     return [
79         periodo_mese(mese_con_offset(oggi, -offset))
80         for offset in range(numero_mesi)
81     ]

```

Questa funzione genera l'elenco dei periodi che dovranno essere elaborati dal programma durante la creazione dei report. Partendo dalla data corrente, oppure da una data fornita esplicitamente, determina automaticamente i mesi da analizzare e per ciascuno di essi costruisce il relativo intervallo temporale completo. In questo modo il software può produrre più report consecutivi in un'unica esecuzione, ad esempio per il mese corrente e per i mesi precedenti, senza richiedere all'utente di specificare manualmente ogni singolo periodo di riferimento.

Definiamo la Funzione "carica_cave():"

```

82 def carica_cave(percorso):
83     mappa = {}
84     with open(percorso, "r", encoding="utf-8") as f:
85         next(f)
86         for riga in f:
87             riga = riga.strip()
88             if not riga:
89                 continue
90             prefisso, cava = riga.split()
91             mappa[prefisso] = cava
92     return mappa

```

Questa funzione si occupa di caricare la tabella di corrispondenza tra i prefissi identificativi degli articoli e le rispettive cave di appartenenza. Leggendo un file di configurazione esterno, costruisce una struttura dati che permette al programma di associare automaticamente ogni articolo alla cava corretta durante le fasi di elaborazione. L'utilizzo di un file separato rende questa associazione facilmente modificabile nel tempo, consentendo di aggiornare o aggiungere nuove corrispondenze senza intervenire direttamente sul codice dell'applicativo.

Definiamo la Funzione "_applica_colonne_dataset():"

```

93 def _applica_colonne_dataset(df, dataset_config):
94     keep_columns = dataset_config.get("keep_columns", [])
95     if keep_columns:
96         colonne = [col for col in keep_columns if col in df.columns]
97         df = df[colonne]
98     rename_columns = dataset_config.get("rename_columns", {})
99     if rename_columns:
100         df = df.rename(columns=rename_columns)
101     return df
102
103

```

Questa funzione ha il compito di adattare il dataset alla struttura prevista dal report finale. In una prima fase mantiene solamente le colonne considerate utili per l'elaborazione, eliminando tutte le informazioni superflue che non dovranno comparire nelle fasi successive. Successivamente applica eventuali rinominazioni dei campi, trasformando i nomi tecnici provenienti dalle sorgenti dati in etichette più leggibili e coerenti con il contesto applicativo. In questo modo il dataset viene normalizzato e preparato per le elaborazioni successive e per la generazione del report finale.

Definiamo la Funzione "_calcola_youth():"

```

104 def _calcola_youth(df, menno):
105     df = df.rename(columns={"data_documento": "Youth"})
106     if menno[0] == "_RANGE_":
107         df["Youth"] = pd.to_datetime(df["Youth"], errors="coerce").dt.strftime("%m/%Y")
108     else:
109         df["Youth"] = menno[0]
110     return df

```

Questa funzione si occupa di generare e valorizzare il campo utilizzato per identificare il periodo di riferimento del report. A seconda della modalità di elaborazione scelta, il valore può essere assegnato in modo uniforme a tutte le righe utilizzando il mese elaborato dal programma oppure essere ricavato direttamente dalle date presenti nei dati originali. In quest'ultimo caso le informazioni temporali vengono convertite in un formato standard mese/anno, garantendo uniformità nella rappresentazione dei periodi e facilitando successive operazioni di raggruppamento, analisi e consultazione dei report generati.

Definiamo la Funzione "_calcola_cava():"

```

111 def _calcola_cava(df):
112     mappa_cave = carica_cave(CONFIG_DIR / "config_cava.txt")
113     df["Cava"] = (
114         df["Codice_Articolo"]
115         .astype(str)
116         .str[:2]
117         .map(mappa_cave)
118         .fillna("CAVA_NON_MAPPATA")
119     )
120     return df

```

Questa funzione assegna automaticamente a ogni articolo la relativa cava di provenienza utilizzando una tabella di corrispondenza definita in un file di configurazione. L'associazione viene effettuata a partire dal prefisso del codice articolo e, nel caso in cui non venga trovata alcuna corrispondenza, viene assegnato un valore che segnala la necessità di aggiornare la mappatura.

Definiamo la Funzione ”_calcola_val_tot():”

```

121 def _calcola_val_tot(df):
122     df["Sconti_Docrig"] = (
123         df["Sconti_Docrig"]
124         .astype(str)
125         .str.replace(".", "", regex=False)
126         .str.replace("EUR", "", regex=False)
127         .str.strip()
128     )
129     df["Sconti_Docrig"] = pd.to_numeric(df["Sconti_Docrig"], errors="coerce").fillna(0)
130
131     df["Val_tot"] = (
132         (
133             df["prezzo_unitario_docrig"]
134             - (df["prezzo_unitario_docrig"] * df["Sconti_Docrig"] / 100)
135             - df["sconti_valore_docrig"]
136             + df["Prezzo_Trasp"]
137         )
138         * df["Quantita"]
139     )
140     return df

```

Questa funzione si occupa di calcolare il valore economico totale associato a ciascuna riga del report. Prima di effettuare il calcolo, normalizza il campo relativo agli sconti, eliminando eventuali caratteri testuali o formati non numerici e convertendo il valore in un numero utilizzabile dal programma. Successivamente determina il valore effettivo della riga considerando il prezzo unitario dell'articolo, lo sconto percentuale applicato, eventuali sconti espressi come valore assoluto, il prezzo di trasporto e la quantità movimentata. Il risultato ottenuto viene inserito in una nuova colonna dedicata, che rappresenta uno degli elementi principali del report perché consente di analizzare non solo le quantità movimentate, ma anche il loro impatto economico complessivo.

Definiamo la Funzione ”applica_campi_calcolati():”

```

141 def applica_campi_calcolati(df, dataset_config, menno):
142     calculated_fields = set(dataset_config.get("calculated_fields", []))
143
144     if "Younth" in calculated_fields:
145         df = _calcola_youth(df, menno)
146     if "Cava" in calculated_fields:
147         df = _calcola_cava(df)
148     if "Val_tot" in calculated_fields:
149         df = _calcola_val_tot(df)
150
151     return df

```

Questa funzione gestisce l'applicazione dei campi calcolati previsti dalla configurazione del dataset. In base alle impostazioni definite, richiama le procedure necessarie per generare automaticamente informazioni che non sono presenti direttamente nelle sorgenti dati, ma che risultano utili per la costruzione del report finale. Tra queste rientrano l'identificazione del periodo di riferimento, l'associazione degli articoli alle rispettive cave e il calcolo del valore economico totale delle movimentazioni. In questo modo il processo di arricchimento del dataset rimane configurabile e può essere esteso nel tempo senza modificare la logica generale dell'applicativo.

Definiamo la Funzione ”_condizione_report()”:

```

152 def _condizione_report(df, regola):
153     colonna = regola.get("column")
154     operatore = regola.get("operator", "eq")
155     if colonna not in df.columns:
156         return pd.Series(False, index=df.index)
157
158     serie = df[colonna]
159     if operatore == "blank":
160         testo = serie.astype("string").str.strip()
161         return serie.isna() | testo.isin(["", "<NA>", "nan", "NaN", "None"])
162
163     if operatore == "not_blank":
164         testo = serie.astype("string").str.strip()
165         return ~(serie.isna() | testo.isin(["", "<NA>", "nan", "NaN", "None"]))
166
167     valore = regola.get("value")
168     if isinstance(valore, (int, float)):
169         numeri = pd.to_numeric(serie, errors="coerce")
170         if operatore == "eq":
171             return numeri.eq(valore)
172         if operatore == "ne":
173             return numeri.ne(valore)
174
175     testo = serie.astype("string").str.strip()
176     valore_testo = str(valore).strip()
177     if operatore == "eq":
178         return testo.eq(valore_testo)
179     if operatore == "ne":
180         return testo.ne(valore_testo)
181
182     raise ValueError(f"Operatore {operatore} non supportato: {operatore}")

```

Questa funzione si occupa di valutare una singola regola di filtro da applicare al dataset. In base alla configurazione ricevuta, verifica l'esistenza della colonna interessata e controlla se i valori presenti soddisfano una determinata condizione, come l'uguaglianza o la differenza rispetto a un valore specifico, oppure la presenza o l'assenza di contenuto all'interno delle celle. La funzione gestisce sia dati numerici sia dati testuali, adattando automaticamente il tipo di confronto da eseguire. Il risultato finale è una condizione logica che identifica le righe del dataset che rispettano i criteri definiti e che potranno quindi essere mantenute o escluse nelle successive fasi di elaborazione del report.

Definiamo la Funzione ”applica_filtr_report()”

```

183 def applica_filtr_report(df, config_report):
184     df_filtrato = df
185     for filtro in config_report.get("drop_rows", []):
186         condizioni = filtro.get("all", [])
187         if not condizioni:
188             continue
189
190         maschera = pd.Series(True, index=df.index)
191         for regola in condizioni:
192             maschera &= _condizione_report(df_filtrato, regola)
193
194     df_filtrato = df_filtrato.loc[~maschera].copy()
195
196     return df_filtrato

```

Questa funzione applica al dataset finale i filtri definiti nella configurazione del report. Per ciascuna regola vengono valutate una o più condizioni che devono essere soddisfatte contemporaneamente e, quando una riga rispetta tutti i criteri specificati, viene esclusa dal risultato finale. Questo meccanismo consente di eliminare automaticamente dati non rilevanti, incompleti o non desiderati, mantenendo nel report soltanto le informazioni utili all'analisi e rendendo il comportamento del programma facilmente modificabile attraverso la configurazione senza intervenire sul codice sorgente.

Definiamo la Funzione ”prepara_base_dati()”

```

197 def prepara_base_dati(menno, sorgente, dataset_config):
198     df = df_format_esegui(base_dir(sorgente), dataset_config, menno)
199     df = applica_colonne_dataset(df, dataset_config)
200     df = applica_campi_calcolati(df, dataset_config, menno)
201     return df

```

Questa funzione coordina la preparazione del dataset che verrà utilizzato per la generazione del report. Inizialmente richiama la procedura di lettura e costruzione dei dati a partire dalle sorgenti configurate, successivamente applica le operazioni di selezione e rinomina delle colonne e infine genera gli eventuali campi calcolati richiesti dalla configurazione. Il risultato è un dataset completo, normalizzato e pronto per le successive fasi di filtraggio, aggregazione ed esportazione.

Definiamo la Funzione "applica_config_report():"

```

202 def applica_config_report(df, config_report):
203     df = applica_filtri_report(df, config_report)
204     group_by = config_report.get("group_by", [])
205     aggregazioni = config_report.get("aggregazioni")
206     df_report = dc.esegui(df, group_by, aggregazioni)
207
208     colonne = config_report.get("columns")
209     if colonne:
210         colonne_presenti = [col for col in colonne if col in df_report.columns]
211         df_report = df_report[colonne_presenti]
212
213     ordinamento = config_report.get("sort_by", group_by)
214     if ordinamento:
215         colonne_ordinamento = [col for col in ordinamento if col in df_report.columns]
216         if colonne_ordinamento:
217             df_report = df_report.sort_values(colonne_ordinamento, kind="stable")
218
219     return df_report

```

Questa funzione applica al dataset tutte le regole definite nella configurazione del report, trasformando i dati preparati nelle informazioni che verranno effettivamente esportate. Inizialmente vengono applicati i filtri che permettono di escludere le righe non rilevanti, successivamente i dati vengono eventualmente raggruppati e aggregati secondo i criteri specificati. Una volta ottenuto il dataset finale, vengono selezionate le sole colonne che dovranno comparire nel report e viene applicato l'ordinamento previsto dalla configurazione. Il risultato è una tabella organizzata e coerente con le esigenze informative del report richiesto, pronta per essere esportata nel file Excel finale.

Definiamo la Funzione "prepara_config_periodo():"

```

220 def prepara_config_periodo(config_report, menno):
221     config_periodo = config_report.copy()
222     config_periodo["periodo"] = menno[0]
223     config_periodo["periodo_file"] = menno[1].strftime("%Y-%m")
224     return config_periodo

```

Questa funzione prepara una versione della configurazione specifica per il periodo in elaborazione. A partire dalla configurazione generale del report, aggiunge le informazioni relative al mese analizzato sia in un formato destinato alla visualizzazione all'interno del report sia in un formato utilizzabile per la generazione del nome del file. In questo modo ogni report prodotto mantiene un riferimento esplicito al periodo a cui si riferisce, facilitando l'organizzazione e l'archiviazione dei risultati nel tempo.

Definiamo la Funzione "periodo_intervallo():"

```

225 def periodo_intervallo(periodi):
226     data_inizio = min(periodo[1] for periodo in periodi)
227     data_fine = max(periodo[2] for periodo in periodi)
228     print(f"Intervallo lettura DBF: {data_inizio} -> {data_fine}")
229     return "__RANGE__", data_inizio, data_fine

```

Questa funzione determina l'intervallo temporale complessivo necessario per la lettura dei dati quando devono essere elaborati più periodi contemporaneamente. A partire dall'insieme dei mesi richiesti, individua automaticamente la data di inizio più lontana e la data di fine più recente, costruendo un unico intervallo di riferimento. Tale informazione viene utilizzata per limitare la lettura dei dati alle sole registrazioni effettivamente necessarie, ottimizzando le prestazioni dell'applicativo e riducendo il volume di informazioni da elaborare.

Definiamo la Funzione "esegui_periodo():"

```

230 def esegui_periodo(menno, nome_report=None, sorgente="rete"):
231     config_report = report_config.carica(CONFIG_DIR / "report_config.json", nome_report)
232     config_report = prepara_config_periodo(config_report, menno)
233     df_base = prepara_base_dati(menno, sorgente, config_report["dataset_config"])
234     df_report = applica_config_report(df_base, config_report)
235     return df_report, config_report

```

Questa funzione coordina l'intero processo di generazione di un singolo report relativo a un determinato periodo. Inizialmente carica la configurazione richiesta, la adatta al mese in elaborazione e prepara il dataset leggendo ed elaborando i dati provenienti dalle sorgenti configurate. Successivamente applica tutte le regole specifiche del report, come filtri, aggregazioni, selezione delle colonne e ordinamenti, ottenendo così il dataset finale pronto per l'esportazione. Al termine restituisce sia i dati elaborati sia la configurazione utilizzata, che verrà impiegata nelle fasi successive di generazione del file Excel.

Definiamo la Funzione "esegui_tutto():"

```

236 def esegui_tutto(nome_report=None, sorgente="rete", numero_mesi=3):
237     risultati = []
238     periodi = periodi_da_elaborare(numero_mesi)
239     if not periodi:
240         return risultati
241
242     config_report_base = report_config.carica(CONFIG_DIR / "report_config.json", nome_report)
243     dataset_config = config_report_base["dataset_config"]
244     df_base = prepara_base_dati(periodo_intervallo(periodi), sorgente, dataset_config)
245
246     for menno in periodi:
247         config_report = prepara_config_periodo(config_report_base, menno)
248         df_periodo = df_base[df_base["Youth"].eq(menno[0])].copy()
249         df_report = applica_config_report(df_periodo, config_report)
250         risultati.append((df_report, config_report))
251     return risultati

```

Questa funzione gestisce l'elaborazione completa di più report mensili in un'unica esecuzione. Inizialmente richiama `periodi_da_elaborare()` per determinare quali mesi devono essere prodotti; successivamente carica la configurazione generale tramite `report_config.carica()` e prepara un unico dataset di base attraverso `prepara_base_dati()`, usando `periodo_intervallo()` per leggere in una sola volta tutti i dati necessari. Una volta ottenuta la base dati complessiva, la funzione scorre i singoli periodi, richiama `prepara_config_periodo()` per adattare la configurazione al mese specifico, seleziona le sole righe appartenenti a quel periodo e applica le regole del report tramite `applica_config_report()`. Alla fine restituisce l'elenco dei report pronti per essere esportati, ciascuno associato alla propria configurazione.

Il *main*: il cuore del programma

```

252 if __name__ == "__main__":
253     try:
254         parser = argparse.ArgumentParser(description="Genera report Excel dal DBF.")
255         parser.add_argument("--report", help="Nome report definito in config/report_config.json")
256         parser.add_argument(
257             "--source",
258             choices=["rete", "esempi"],
259             default="rete",
260             help="Sorgente DBF. In produzione usa rete: i DBF sono nella cartella ICR; i DBF restano nel progetto."
261         )
262         parser.add_argument(
263             "--months",
264             type=int,
265             default=3,
266             help="Numero di mesi da generare, includendo il mese corrente."
267         )
268         parser.add_argument(
269             "--no-open",
270             action="store_true",
271             help="Non aprire Excel al termine. Utile per esecuzione pianificata."
272         )
273         args = parser.parse_args()
274         config.verifica = report_config.carica(CONFIG_DIR / "report_config.json", args.report)
275         verifica_percorsi(args.source, config.verifica["dataset_config"])
276         percorsi_creati = []
277         for df, config_report in esegui_tutto(args.report, args.source, args.months):
278             print("Esportazione del file Excel in corso...")
279             percorso = df.to_excel.esegui(df, config_report)
280             percorsi_creati.append(percorso)
281         if percorsi_creati and not args.no_open:
282             os.startfile(percorsi_creati[0])
283         except Exception as exc:
284             print(f"ERRORE: {exc}")
285             raise SystemExit(1)

```

Questa sezione rappresenta il punto di ingresso principale dell'applicativo, cioè la parte che viene eseguita quando il programma viene avviato direttamente dall'utente o tramite file batch. Il blocco iniziale permette di distinguere il modulo utilizzato come libreria dal programma eseguito come applicativo autonomo; in questo secondo caso viene avviata l'intera procedura di generazione dei report. Per prima cosa viene predisposto il sistema di argomenti da terminale, attraverso il quale l'utente può specificare il report da generare, la sorgente dei dati, il numero di mesi da elaborare e la modalità di apertura finale dei file Excel. L'argomento relativo al report consente di scegliere una configurazione presente nel file `report_config.json`, mentre l'argomento relativo alla sorgente permette di decidere se utilizzare i dati reali presenti nella cartella di rete aziendale oppure i file locali di esempio. Il parametro relativo ai mesi stabilisce quanti periodi devono essere generati, includendo il mese corrente, mentre l'opzione che impedisce l'apertura automatica del file finale risulta particolarmente utile quando il programma viene eseguito in modo pianificato da Windows.

Una volta acquisiti gli argomenti, il programma carica la configurazione richiesta richiamando `report_config.carica()`, in modo da recuperare sia le impostazioni del report sia quelle del dataset associato. Prima di procedere con l'elaborazione vera e propria, viene eseguita una verifica dei percorsi tramite `verifica_percorsi()`, così da controllare che i file DBF necessari e i file di configurazione siano effettivamente disponibili nella posizione prevista. Questo passaggio è importante perché consente di individuare subito eventuali problemi legati a file mancanti, percorsi errati o assenza della connessione alla cartella di rete.

Dopo la fase di controllo, il programma richiama `esegui_tutto()`, che rappresenta la funzione incaricata di elaborare tutti i periodi richiesti. Essa produce, per ciascun mese, un DataFrame già filtrato, trasformato, aggregato e conforme alla configurazione del report. Per ogni risultato ottenuto, il programma avvia la fase di esportazione richiamando `df.to_excel.esegui()`, che genera materialmente il file Excel, applica la formattazione, crea eventuali fogli di riepilogo e restituisce il percorso del file prodotto. Tutti i percorsi dei report creati vengono memorizzati in un elenco, così da poter gestire successivamente l'apertura automatica del primo file generato.

La parte finale del blocco gestisce infatti il comportamento dell'applicativo dopo l'esportazione. Se almeno un report è stato creato e l'utente non ha richiesto esplicitamente di evitare l'apertura automatica, il programma apre il primo file Excel prodotto, facilitando il controllo immediato del risultato. L'intera procedura è racchiusa in una gestione degli errori che intercetta eventuali eccezioni, stampa un messaggio esplicativo e termina il programma con un codice di uscita non valido. Questa scelta rende il comportamento più adatto anche all'esecuzione automatica tramite attività pianificata, perché consente di distinguere chiaramente un'esecuzione completata correttamente da una terminata con errore.

3.5 Cartella docs

La cartella `docs` contiene la documentazione tecnica e operativa del progetto.

Il suo scopo è raccogliere tutte le informazioni necessarie per comprendere il funzionamento del software, facilitarne la manutenzione nel tempo e supportare eventuali futuri sviluppatori o operatori incaricati della gestione del sistema.

In questa cartella sono presenti un manuale contenente procedure operative e indicazioni utili per la configurazione e la messa in produzione del programma, e questo articolo, nel quale è descritta l'architettura dell'applicativo,.

3.5.1 File: `Manuale_Software_Creazione_Report.pdf`

Questo documento rappresenta il manuale tecnico e operativo dell'applicativo.

Al suo interno vengono descritti lo scopo del programma, la struttura delle cartelle, le modalità di esecuzione in ambiente di sviluppo e produzione, il funzionamento generale della pipeline di elaborazione e il ruolo dei principali moduli Python.

Sono inoltre presenti istruzioni dettagliate per la configurazione dei report, la creazione di nuovi dataset, la gestione dei campi calcolati, la risoluzione dei problemi più comuni, l'installazione dell'attività pianificata di Windows e la rigenerazione dell'eseguibile distribuito in produzione.

Il manuale costituisce quindi il principale riferimento operativo per l'utilizzo e la manutenzione del software.

Chapter 4

Cartella release

La cartella **release_*** é il pacchetto finale, pronto all'uso. Al suo interno si trovano l'eseguibile dell'applicativo, la cartella **config** contenente le configurazioni dei report, la cartella **export** destinata ad ospitare i file Excel generati, la cartella **logs** utilizzata per registrare le esecuzioni del programma e gli eventuali errori, oltre ai file batch necessari per l'avvio del software e per la configurazione dell'esecuzione automatica giornaliera. La struttura è stata progettata per consentire il funzionamento completo dell'applicativo senza richiedere ulteriori installazioni o configurazioni manuali.

4.1 Cartella config

La cartella config raccoglie tutti i file di configurazione necessari al funzionamento del software. Grazie a questi file è possibile modificare il comportamento dell'applicativo senza intervenire direttamente sul codice sorgente, rendendo il sistema più flessibile e facilmente adattabile a nuovi report o nuove sorgenti dati.

4.1.1 File: report_config.json

Il file **report_config.json** rappresenta il centro della configurazione dell'intero applicativo. Attraverso questo file è possibile definire le sorgenti dati, le relazioni tra le tabelle, le trasformazioni da applicare e le caratteristiche dei report da generare senza modificare direttamente il codice Python.

Configurazione generale

```
1 {  
2   "default_report": "cave_mensile",  
3   "datasets": {
```

In questa sezione viene definito il report predefinito e viene aperta la struttura che contiene tutti i dataset disponibili. Qualora si volesse aggiungere una nuova tipologia di report, questa rappresenta la prima sezione da modificare.

Definizione delle sorgenti dati

```

1  "sources": {
2  | "doctes": {...},
3  | "docrig": {...},
4  | "vincfatt": {...},
5  | "canvds": {...},
6  | "cdccos": {...},
7  | "cdccric": {...},
8  | "maggrp": {...},
9  | "anagrafe": {...}
10 | }

```

Questa sezione definisce tutte le tabelle DBF utilizzate dal programma. Per ciascuna sorgente vengono specificati il file da leggere, le colonne necessarie e gli eventuali filtri da applicare durante l'importazione. Per creare un nuovo report che richieda dati aggiuntivi è possibile inserire qui nuove sorgenti o ampliare quelle esistenti.

Relazioni tra le tabelle

```

1  "joins": [
2  | {
3  | | "...",
4  | | "...",
5  | | ...

```

In questa parte vengono definite le relazioni tra le diverse tabelle. Il sistema utilizza tali informazioni per ricostruire un dataset unico partendo da più sorgenti distinte. Per integrare nuove informazioni provenienti da altre tabelle è necessario aggiungere qui i relativi collegamenti.

Selezione e rinomina delle colonne

```

1  "keep_columns": [...],
2  |
3  | "rename_columns": {
4  | | ...
5  | }

```

Questa sezione stabilisce quali campi mantenere dopo le operazioni di unione e come rinominarli. In questo modo il dataset finale contiene soltanto le informazioni realmente utili e presenta nomi facilmente comprensibili dagli utenti finali.

Campi calcolati

```

1  "calculated_fields": [
2  | "founth",
3  | "Cava",
4  | "Val_tot"
5  | ]

```

Qui vengono definiti i campi che non esistono direttamente nei database ma che devono essere calcolati dal programma durante l'elaborazione. Nel caso specifico vengono generati il periodo di riferimento, la cava associata all'articolo e il valore economico totale della movimentazione.

Configurazione del report

```

1  "reports": {
2  | "cave_mensile": {
3  | | ...
4  | | }
5  | }

```

Questa sezione contiene la configurazione vera e propria del report. Ogni report può utilizzare uno specifico dataset e definire regole di aggregazione, ordinamento ed esportazione differenti.

Regole di aggregazione

```

1 "group_by": [
2   ...
3 ],
4 "aggregations": {
5   ...
6 }
7

```

Le informazioni presenti in questa sezione determinano il livello di dettaglio del report finale. Modificando i campi di raggruppamento e le operazioni di aggregazione è possibile ottenere report sintetici oppure analitici senza intervenire sul codice sorgente.

Layout del report Excel

```

1 "columns": [...],
2 "column_widths": {...},
3 "wrap_columns": [...],
4 "number_formats": {...}
5

```

Questa sezione controlla l'aspetto del file Excel generato. Vengono definite le colonne da visualizzare, il loro ordine, la larghezza, i formati numerici e le eventuali colonne che devono utilizzare il ritorno automatico a capo.

Fogli di riepilogo

```

1 "summaries": [
2   {
3     ...
4   }
5 ]

```

L'ultima sezione permette di definire uno o più fogli riepilogativi che verranno generati automaticamente insieme al report principale. Tali fogli consentono di ottenere viste aggregate dei dati senza dover utilizzare strumenti di analisi aggiuntivi all'interno di Excel.

Descrizione delle principali sorgenti dati

Le tabelle utilizzate dal software derivano dal database FoxPro aziendale e contengono informazioni complementari tra loro. Per la tabella relativa al report cave, i dati sono stati ricavati da:

- DOCTES: contiene le testate dei documenti;
- DOCRIG: contiene le righe associate ai documenti;
- ANAGRAFE: contiene clienti e fornitori;
- MAGGRP: contiene i gruppi di magazzino;
- VINCFATT: contiene collegamenti relativi ai processi di fatturazione;
- CANVDS: contiene informazioni aggiuntive sulle movimentazioni;
- CDCCOS: contiene dati relativi ai centri di costo;
- CDCRIC: contiene dati relativi ai centri di ricavo.

L'unione di queste sorgenti consente al software di costruire un dataset completo e coerente per la generazione dei report.

4.1.2 File: config_cava.txt

```

1 tipo      cava
2 SL        San_Leo
3 SC        San_Carlo
4 CO        San_Carlo
5 CE        Cesena
6 PA        Palazzina
7 MA        Manzona
8 SA        Santarcangelo
9 VE        Verucchio
10 LE       Lercetti
11 TA       Talamello
12 DEPOSITO Deposito

```

Il file `config_cava.txt` contiene la tabella di corrispondenza tra i prefissi presenti nei codici articolo e le relative cave di provenienza. Questa configurazione permette al programma di assegnare automaticamente ogni articolo alla cava corretta senza dover inserire manualmente tale informazione nel codice Python. In caso di modifiche future, come l'aggiunta di una nuova cava o la variazione di un prefisso, sarà sufficiente aggiornare questo file di testo.

4.2 Cartella export

La cartella `export` contiene i file Excel generati dall'applicativo al termine dell'elaborazione. Nel progetto di sviluppo sono presenti alcuni report di esempio, utilizzati per verificare il corretto funzionamento del programma e controllare la struttura dell'output prodotto. Nella versione distribuita, invece, questa cartella verrà creata e popolata automaticamente all'interno della cartella `release_fast`, dove gli utenti troveranno i report generati dall'eseguibile.

4.2.1 File di esempio

I file `report_2026-04.xlsx`, `report_2026-05.xlsx` e `report_2026-06.xlsx` rappresentano esempi di output mensile prodotti dal software. Essi mostrano la struttura finale del report, comprensiva dei dati elaborati, delle colonne configurate, della formattazione Excel e degli eventuali fogli di riepilogo. In ambiente di produzione, file analoghi verranno creati automaticamente nella cartella `export` del pacchetto distribuito.

4.3 Cartella _internal

La cartella `_internal` viene generata automaticamente durante la creazione dell'eseguibile e contiene tutte le librerie, i moduli compilati e le dipendenze necessarie al funzionamento dell'applicativo. Sebbene non venga utilizzata direttamente dall'utente finale, essa rappresenta un componente essenziale del pacchetto distribuito, poiché ospita tutti gli elementi richiesti dall'eseguibile per eseguire le operazioni di lettura dei dati, elaborazione dei report e generazione dei file Excel. La cartella è gestita automaticamente dal processo di build e non richiede alcun intervento da parte dell'utente; per questo motivo non deve essere modificata né rimossa dalla distribuzione finale, in quanto la sua assenza comprometterebbe il corretto funzionamento del software.

Chapter 5

Utilizzo del Software

5.1 Avvio manuale

Per generare i report è sufficiente eseguire il file batch:

```
run_report_periodico.bat
```

Durante l'esecuzione il programma:

1. verifica la presenza dei file necessari;
2. controlla la disponibilità della rete aziendale;
3. legge i dati dai file DBF;
4. genera i report configurati;
5. salva i risultati nella cartella **export**.

Al termine dell'elaborazione i file Excel saranno disponibili per la consultazione.

5.2 Esecuzione automatica

Il software è predisposto per l'esecuzione automatica mediante l'Utilità di Pianificazione di Windows.

L'installazione dell'attività pianificata può essere effettuata eseguendo il file:

```
installa_attivita_giornaliera.bat
```

Una volta configurata, l'attività avvierà automaticamente il programma secondo la frequenza definita dall'amministratore del sistema, senza richiedere l'intervento degli operatori.

Chapter 6

Risoluzione dei problemi

6.1 File DBF non trovati

Verificare che la cartella di rete aziendale sia raggiungibile e che l'utente disponga dei permessi necessari per la lettura dei dati.

6.2 File di configurazione mancanti

Verificare la presenza dei seguenti file all'interno della cartella **config** (per Report Cave):

- **report_config.json**
- **config_cava.txt**

6.3 Report non generato

Verificare che il file Excel non sia aperto da un altro utente. In caso contrario il software tenterà automaticamente di generare una copia del report utilizzando un nome differente.

6.4 CAVA_NON_MAPPATA

Questo messaggio indica che il prefisso di un articolo non è presente nel file **config_cava.txt**. Per risolvere il problema è necessario aggiornare la configurazione aggiungendo la relativa associazione.

6.5 Errore durante l'avvio

Verificare la presenza di tutti i file contenuti nella cartella **release** ed evitare la rimozione della cartella **_internal**, necessaria per il funzionamento dell'applicativo.

Chapter 7

Manutenzione ed estensioni future

L'architettura del software è stata progettata per consentire l'introduzione di nuove tipologie di report senza modificare il codice sorgente. Nella maggior parte dei casi è sufficiente intervenire sui file di configurazione presenti nella cartella `config`, definendo nuove sorgenti dati, nuove regole di aggregazione o nuovi layout di esportazione.

Questa caratteristica consente al sistema di adattarsi facilmente all'evoluzione delle esigenze aziendali e riduce significativamente i costi di manutenzione futura.

Tra i possibili sviluppi futuri rientrano l'integrazione con ulteriori sorgenti dati, la generazione automatica di report PDF, l'invio tramite posta elettronica e la realizzazione di dashboard interattive per la consultazione dei dati.